TRANSFUZZ: Cross-Language Transfer of Exploit-Derived Seeds for PHP Interpreter Fuzzing

Abstract

Interpreters implemented in memory-unsafe C/C++ are a high-value target for memory-corruption attacks with their vulnerabilities enabling remote code execution, sandbox escape, and denial-of-service across plugin ecosystems, user-submitted content platforms, and shared hosting. Yet, testing effort is distributed unevenly. JavaScript engines receive extensive fuzzing and a large public corpus of exploit PoCs, while PHP, despite powering a comparable share of Internet infrastructure, remains critically under-tested. We find that this disparity can be addressed by transferring exploit scaffolding across languages. Specifically, we show that JavaScript PoCs, when translated into PHP, carry over rich structures (loops, nested calls, extreme values, and exception handlers) that expose PHP interpreter vulnerabilities which state-of-the-art fuzzers struggle to reach, even at comparable basic block coverage. We present TRANSFUZZ, a cross-language fuzzing system that realizes this finding. TRANSFUZZ uses large language models (LLMs) to translate JavaScript PoCs into PHP seeds, enriches them via LLM-guided splicing with PHP-native auxiliary seeds, and employs ancestry-aware scheduling and LLM-based repair to sustain throughput. Evaluated on three official PHP interpreter versions (8.3.9, 8.4.1, and 8.5.0-dev), TRANSFUZZ uncovers 16 security-relevant memory-safety vulnerabilities, including five use-after-frees and four invalid memory accesses. 39 of the 50 findings were previously unknown and are acknowledged by the PHP development team. Our results demonstrate that exploit corpora from mature ecosystems can provide effective seeds for vulnerability discovery in under-tested interpreters.

1 Introduction

Language interpreters are a high-value target for memory-corruption attacks. Widely implemented in memory-unsafe C/C++, interpreters can execute attacker-influenced code across web services, plugin ecosystems, shared hosting, and user-submitted-content platforms, leading to memory-safety vulnerabilities repeatedly being exploited for remote code execution [1], sandbox escape [2, 3], and denial-of-service [4]. Consequently, significant research effort has been devoted to developing increasingly sophisticated techniques for interpreter fuzzing [5].

Finding interpreter vulnerabilities, however, requires more than generating syntactically valid programs or maximizing code coverage. Many memory-safety faults manifest only after complex sequences of state transitions induced by nested callbacks, loops, exception handling, or object interactions. Producing programs with these structures remains difficult. Grammar-based fuzzers can generate individual language constructs, but rarely compose them into the intricate structures required by state-dependent bugs [6–9]. Mutation-based approaches inherit the structural limitations of their seed corpora, while recent data-flow-guided techniques improve mutations without addressing the scarcity of structurally rich starting inputs [10]. Thus, interpreter fuzzing remains constrained by the availability of seeds that encode complex, security-relevant execution patterns.

We observe that such seeds exist but are largely confined to their native ecosystems. Historical exploit proof-of-concepts (PoCs) capture adversarially discovered exploit scaffolding: nested calls, loops, object mutations, and exception paths that drive an interpreter through unusual states. Mature ecosystems such as JavaScript have accumulated large corpora of these programs through years of browser-security research and bug bounty programs, whereas less studied interpreters have substantially smaller exploit corpora.

This raises the central question of this work: *can exploit PoCs in one ecosystem serve as effective seeds in a different ecosystem?* Exact vulnerability semantics are unnecessary. For example, a JavaScript PoC that triggers a V8 use-after-free may depend on APIs or memory-management behavior with no PHP equivalent. Instead, we hypothesize that higher-level execution patterns can survive translation and stress stateful mechanisms in another interpreter.

We investigate this hypothesis using JavaScript→PHP as a deliberately selected study setting. JavaScript and PHP expose

sufficiently similar high-level abstractions, including dynamic typing, mutable objects, and permissive execution behavior, to make structural transfer plausible. Simultaneously, their security-testing ecosystems are highly asymmetric. JavaScript engines have accumulated extensive public exploit corpora, while the official PHP interpreter repository contains only 52 CVE PoCs. JavaScript→PHP provides a concrete setting in which to determine whether exploit scaffolding can remain useful for vulnerability discovery across a language boundary.

Our results show that it can. Recent work has shown that large language models (LLMs) perform semantics-preserving translation reliably [11–13], and our LLM-translated JavaScript PoCs retain 77% of the source structural features measured by our preservation metric. In seven independent 24-hour experiments, our approach achieves basic-block coverage comparable to FlowFusion, the state-of-the-art data-flow PHP fuzzer (41.7K versus 41.5K blocks), yet discovers 4.71 unique bugs per run compared with FlowFusion's 1.28 representing a 3.6× improvement. This result suggests that aggregate coverage alone does not capture the value of exploit-derived execution structure for stateful interpreter fuzzing.

Practically, implementing such a system is non-trivial. We identify four challenges. (1) Mutation-based fuzzers rely on effective seed corpuses, but PHP's repository contains a limited number of exploit-grade PoCs. (2) There are PHP-specific idioms which are not represented in JavaScript corpuses. (3) LLM inference is slower than traditional fuzzing. (4) The random nature of mutation tends to produce malformed seeds.

To realize this idea, we present TRANSFUZZ, a cross-language fuzzing system that translates JavaScript exploit PoCs into PHP seeds, combines them with PHP-native auxiliary programs, prioritizes promising descendants using execution feedback, and repairs malformed generated inputs using interpreter diagnostics. Across PHP `8.3.9`, `8.4.1`, and `8.5.0-dev`, TRANSFUZZ discovers 50 bugs, including 39 findings previously unknown to the PHP development team.

Why Scaffolding Transfer Works Mechanistically. A JavaScript exploit PoC encodes an *allocator state sequence* that the interpreter must execute for the bug to manifest. Nested getters trigger during operations such as `concat()` or `array_merge()`, inserting side-effecting allocations in what the optimizer assumes is a pure traversal. Loop bodies with concretely computed iteration counts drive realloc boundaries, so the buffer the interpreter resizes is the one whose size was set iterations earlier. Fiber and generator suspensions freeze a reference-counted object with its captured closures intact, and mutation of that object by the parent context opens the inter-object use-after-free window the bug conditions on. Memory-safety bugs in interpreters are faults of *sequencing* over shared allocator, refcount, and object-table states; PoCs that previously triggered V8 UAFs carry exactly this sequencing. Translation into PHP transfers the sequence because PHP and JavaScript are two compatible interpreter ecosystems. The transferred structures need not preserve the source interpreter's exact memory-management semantics. Rather, they preserve higher-level execution patterns (such as nested callbacks, repeated allocations, object mutation, and suspension/resumption) that can induce analogous stress on stateful memory-management mechanisms in the target interpreter.

Contributions. We make the following contributions:

- **Scientific Contribution.** We formulate and empirically investigate the idea that historical exploit PoCs contain execution scaffolding that can remain useful across compatible interpreter ecosystems. In the JavaScript→PHP setting, translated PoCs preserve substantial source structure and enable significantly higher bug discovery than existing PHP fuzzers despite comparable aggregate basic-block coverage.
- **System.** We present TRANSFUZZ, a cross-language fuzzing system that realizes this finding. TRANSFUZZ combines LLM-based JavaScript-to-PHP translation, LLM-guided splicing with PHP-native auxiliary seeds, ancestry-aware scheduling that tracks cross-language seed provenance, and LLM-based repair to sustain throughput.
- **Security Impact.** On three official PHP interpreter versions (`8.3.9`, `8.4.1`, `8.5.0-dev`), TRANSFUZZ discloses *16 security-relevant memory-safety vulnerabilities* (5 use-after-frees, 7 memory leaks, 4 invalid memory accesses) confirmed under our threat model, plus 34 additional bugs; 39 of the 50 findings were previously unknown to the PHP security team and have been responsibly disclosed and acknowledged.

2 Background and Motivation

Interpreter fuzzing plays a central role in detecting security vulnerabilities in programming language runtimes. Fuzzers targeting interpreters typically rely on one of two main strategies: grammar-based input generation or mutation-based fuzzing using seed corpora. Grammar-based fuzzers generate syntactically valid test cases using handcrafted grammars [6, 7], while mutation-based fuzzers [16, 17] mutate existing inputs to explore new program paths. Both approaches require significant manual effort, either in building accurate grammars or curating and maintaining effective seed sets. PoCs in particular are valuable as seeds because they often target corner cases, increasing the likelihood of uncovering vulnerabilities. However, PoC availability is limited in the case of poorly supported languages. For PHP, security efforts have largely focused on PHP applications rather than the interpreter itself [18–20]. Consequently, the official PHP interpreter repository contains only 52 CVE PoCs [21]. Consequently, interpreters for languages with less mature fuzzing infrastructure, such as PHP, remain under-tested compared to better-supported languages like JavaScript.

Table 1: Structural and Semantic Similarities Between PHP and JavaScript [14, 15]

Category	JavaScript	PHP
Typing	Dynamically typed	Dynamically typed
Object Model	Prototype-based objects	Class and associative-array–based objects
Execution Model	Interpreted (V8, SpiderMonkey)	Interpreted (Zend Engine)
Memory-Safety	Implemented in C++	Implemented in C
Error Handling	Loosely typed errors, runtime exceptions	Loosely typed errors, runtime exceptions
Code Injection	'eval()', dynamic property access, reflection	'eval()', 'include', 'variable variables', reflection
Type Coercion	Implicit coercion common (e.g., '[] + {}')	Implicit coercion common (e.g., '"1" + 2' $\rightarrow$ 3)

Language Similarities. JavaScript and PHP, despite being developed for different application domains (client-side vs. server-side scripting), share several important semantic and structural features. As illustrated in Table 1, both are dynamically typed, support associative arrays and objects with mutable structures, permit runtime code evaluation through constructs such as `eval()`, exhibit permissive error-handling behavior, and rely on memory-unsafe C/C++ backends (V8 for JavaScript, Zend Engine for PHP). These shared features imply that programs written in one language may exhibit behavioral patterns that meaningfully transfer to the other. If JavaScript PoC control and data flow constructs are preserved under translation into PHP, the resulting code may likewise trigger deep semantic behaviors in the PHP interpreter.

Why Prior PHP Fuzzers Miss State-Dependent Bugs. The two dominant PHP fuzzing paradigms each fail in a structurally distinct way on this class of bug. Grammar-based fuzzers [6–8] synthesize inputs from a handcrafted production grammar, but the grammar specifies surface syntax (statements, expressions, operator trees), not the deeply nested combinations that deep bugs require (e.g., a getter callback fired from a `concat`-family primitive, wrapped in a `foreach` whose body mutates the collection, invoked from a generator body resumed through a Fiber suspension). A grammar can sample each construct independently, but the probability of drawing such a specific three- or four-level nesting that lands on a Zend UAF trigger is vanishingly small. Data-flow fuzzers [10] take a different route: they mutate existing PHP programs by rewiring taint- and value-flow edges, which generates new data-flow states but cannot introduce new control-flow edges. The corpus compounds the problem: the PHP regression test suite, while containing over 20K cases, is reported to be 96% control-flow-free [10], and mutations inherit this flat structure. Transferred JavaScript PoCs are a qualitatively different corpus where seeds are adversarially constructed to trigger memory-safety bugs in a related interpreter, structurally deep, and encoding exactly the control-flow interactions that a data-flow fuzzer cannot introduce.

Translation Oracle. A cross-language fuzzing oracle does not need to enforce one-to-one correspondence; it suffices to preserve the features that make the original seed valuable (control structures, data operations, and object operations). We quantify preservation via a structural preservation score (SP) over a feature taxonomy $\mathcal{F}$ spanning control, data, and object features, with $c_p(f)$ the count of feature f in program p:

$$\mathrm{SP}(p_{\mathrm{js}}, p_{\mathrm{php}}) = \frac{\sum_{f\in\mathcal{F}} \min(c_{p_{\mathrm{js}}}(f), c_{p_{\mathrm{php}}}(f))}{\sum_{f\in\mathcal{F}} c_{p_{\mathrm{js}}}(f)} \tag{1}$$

Prior work has shown LLMs can provide high-fidelity translations [11–13]. We instantiate this oracle with an LLM and report source-feature retention (SP) of 77% in §6 (RQ4). SP quantifies how much of the source program's structure is preserved; by construction, the denominator is over the source program only, so SP does not penalize PHP-side additions (e.g., auxiliary scaffolding). This matches our design intent where we want to preserve JS structure while allowing PHP-native enrichment.

3 Threat Model and Scope

Goal and Intended Use. TRANSFUZZ is a *vulnerability-discovery* system rather than an exploitation framework. Its intended users are interpreter developers, security researchers, and other benign users who proactively test PHP implementations for memory-safety defects before such defects can be exploited in deployed systems. Accordingly, operating TRANSFUZZ does not require an attacker to already possess code-execution capability on a victim server. In our setting, the tester has legitimate access to the target PHP interpreter and executes generated test programs locally in an isolated environment with sanitizers and other instrumentation enabled. TRANSFUZZ *generates inputs that expose erroneous interpreter behavior; constructing end-to-end exploits or chaining vulnerabilities is outside the scope of this work.*

Security Relevance of Discovered Vulnerabilities. Although vulnerability discovery itself does not assume an active attacker, we assess the security relevance of discovered bugs under deployments in which an adversary can influence data or PHP code processed by the interpreter. Such influence arises naturally in web-facing PHP deployments, including plugin ecosystems, user-submitted-content platforms, shared hosting, and applications that dynamically construct or evaluate PHP code [22–26]. In these environments, the PHP interpreter constitutes part of the trusted computing base. A memory-safety error reachable through adversary-influenced execution can

undermine availability or, depending on the primitive and surrounding state, potentially compromise interpreter isolation. Attacker influence can reach the interpreter at different levels. Historical vulnerabilities illustrate three such paths.

- **P1: Attacker-controlled PHP execution.** Cloud service providers or vulnerabilities in surrounding applications can allow an adversary to supply PHP code for interpretation. CVE-2024-4577 [27], a PHP-CGI argument-injection vulnerability, enabled attacker-controlled PHP execution on affected servers.
- **P2: Application-mediated code generation.** Some applications intentionally translate user-controlled expressions into PHP code. Vulnerabilities in these translation layers can expose the interpreter to attacker-influenced programs. CVE-2026-23836 [28] involved insufficiently sanitized code generation in HotCRP's formula engine.
- **P3: Attacker-controlled values reaching interpreter primitives.** Attacker-controlled values may reach built-in functions and interpreter subsystems through ordinary application inputs. Interpreter bugs triggered by such values can become application-level security vulnerabilities. CVE-2024-11236 [29] demonstrates how an integer overflow in PHP can contribute to an out-of-bounds memory write.

Vulnerabilities discovered by TRANSFUZZ are primarily P1-reachable. As illustrated by Figure 1, P1 occurs in cloud computing environments where servers running PHP interpreters allow users to upload PHP applications for websites or function-as-a-service code. Given the ability to execute arbitrary PHP code, P1 gives the attackers the strongest capability. Enabled through configuration files, the PHP interpreter provides comprehensive sandboxing capabilities to restrict resource usage, control user permissions, and disable sensitive built-in functions (e.g., `system()`).

Scope of Security Assessment. Our objective is to identify and characterize vulnerability primitives, not to construct weaponized exploits. We therefore do not attempt to generate shellcode, privilege-escalation payloads, or end-to-end sandbox escapes, nor do we scan or attack remote systems. For each finding, we instead triage and characterize the resulting primitive, such as use-after-free, invalid memory access, or sustained resource exhaustion. These properties establish the security relevance of the finding, while the feasibility of turning a particular memory-corruption primitive into arbitrary code execution depends on the application context.

Historical Precedent. The security relevance of this threat surface is well-established. Memory-corruption vulnerabilities in language interpreters have previously enabled denial-of-service, sandbox escape, and arbitrary code execution [2, 30]. In the PHP ecosystem specifically, approximately 87% of reported CVEs over the last decade involve memory-corruption issues [31]. These historical cases motivate proactively finding interpreter vulnerabilities before attackers weaponize them. TRANSFUZZ is designed specifically for this defensive purpose.

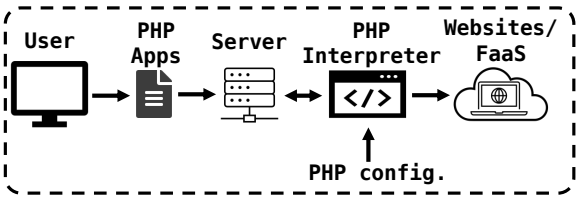


Figure 1: Workflow of cloud-hosted PHP environments.

4 Detailed Design

This section presents the design of TRANSFUZZ. We first summarize the four core challenges in cross-language fuzzing and the four insights that drive our response, then describe the architecture (§4.2), and finally detail the translation and mutation modules (§4.3, §4.4).

4.1 Challenges and Insights

Turning cross-language PoC transfer into a practical fuzzer raises four challenges (C1–C4), each of which motivates one of TRANSFUZZ's design insights (I1–I4).

C1: Initial-corpus scarcity in the target language. A mutation-based fuzzer is only as strong as its seed corpus, yet PHP's upstream repository contains only 52 CVE PoCs, and most PHP security research has targeted applications rather than the interpreter. In contrast, JavaScript engines have accumulated thousands of public PoCs through years of browser bug bounties. Manually porting each PoC is infeasible and would not scale to the size of corpus a modern fuzzing campaign requires.

I1: LLM-based cross-language seed translation. We observe that modern LLMs can translate small programs between syntactically similar languages while preserving structural properties such as loops, exception handlers, and object-usage patterns. We therefore use an LLM to translate each JavaScript PoC into a syntactically valid PHP seed. The translated seed need not be semantically equivalent to the original. For fuzzing, it suffices that the translation preserves *bug-triggering scaffolding*: the control-flow edges, call sequences, and object patterns that made the original PoC valuable. This yields hundreds of structurally rich PHP seeds at essentially zero manual cost, resolving PHP's corpus scarcity with JavaScript's abundance.

C2: Language-specific API and idiom gaps. Even a structurally faithful translation will invoke APIs that do not exist in the target language. JavaScript idioms like `Object.create()` and V8-specific intrinsics (e.g., `%OptimizeFunctionOnNextCall`) have no PHP counterpart; conversely, PHP-native constructs like `__get()` magic

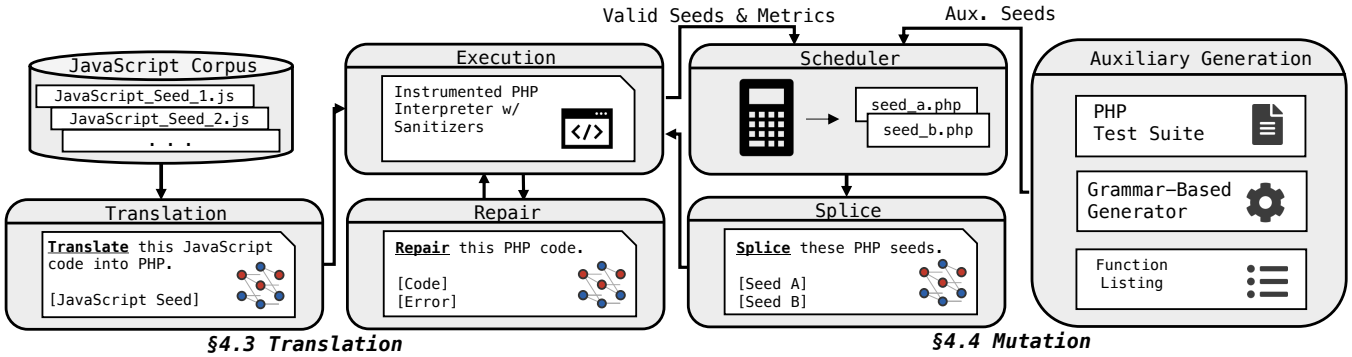


Figure 2: Overview of TRANSFUZZ.

methods, `array_diff()`, `preg_replace_callback()`, and `Fiber` objects are absent in JavaScript PoCs. A corpus of pure translations therefore under-explores PHP-specific interpreter paths that host many of the target's memory-safety bugs.

I2: LLM-guided splicing with PHP-native auxiliary seeds. We address the gap by introducing an auxiliary seed pool drawn from three complementary sources: a grammar-based generator tailored to extreme constants and deeply nested calls, LLM-generated usages of randomly selected PHP built-in functions, and the official PHP regression test suite (over 20,000 cases). During mutation, an LLM splices translated seeds with auxiliary seeds, embedding PHP-specific idioms within the transferred scaffolding. The result is a hybrid seed that combines JavaScript complexity with PHP-native interpreter coverage.

C3: LLM inference is expensive relative to traditional fuzzing. LLM calls are orders of magnitude slower than a grammar-based mutator or a byte-flip pass. Plainly issuing an LLM call for every splicing opportunity would reduce throughput below what is useful at 24-hour campaign scale. Worse, not every seed is equally worth mutating: a spliced result derived from several generations of prior splicing may have drifted away from the bug-triggering scaffolding that made its ancestor useful.

I3: Ancestry-aware scoring and scheduling. We assign each seed a composite score that combines basic-block coverage, a crash anomaly bit, a token-length penalty, and an *ancestry* term that tracks how many JavaScript-translated ancestors the seed descends from. The scheduler preferentially allocates energy to seeds whose ancestry carries many cross-language contributions, preserving control-flow scaffolding. Seeds whose token length approaches the model's context limit are down-weighted so that LLM calls remain within the reliable operating range of the model.

C4: Stochastic mutation produces malformed seeds. LLM-driven splicing frequently produces syntactically or semantically invalid PHP (undeclared identifiers, ill-formed regular expressions, type errors in coerced contexts). Discarding every malformed seed wastes the LLM calls spent producing it and throws away valuable structural mutations that are one small edit from validity.

I4: LLM-based repair loop guided by interpreter diagnostics. When the instrumented PHP interpreter rejects a seed, TRANSFUZZ feeds the interpreter's diagnostic output back to the LLM along with the offending seed and requests a minimal repair. The loop iterates up to five times per seed; repaired seeds rejoin the corpus with their lineage preserved, while seeds that exceed the token budget or repeatedly fail are discarded. The repair loop recovers a non-trivial share of otherwise-wasted LLM effort (§6) and its absence is one of the largest contributors to performance degradation in our ablations.

4.2 TRANSFUZZ Overview

Figure 2 shows TRANSFUZZ's architecture, which consists of two modules: cross-language translation and feedback-driven mutation.

1. **Cross-Language Translation (§4.3):** JavaScript PoCs are translated into PHP using LLMs guided by carefully designed prompt templates (I1). Seeds are executed within an instrumented PHP interpreter to collect runtime feedback. Seeds that fail to execute or trigger errors are passed through an LLM-based repair loop (I4).
2. **Feedback-Driven Mutation (§4.4):** Translated seeds are evaluated by an ancestry-aware scheduler (I3) to identify high-potential inputs. Seeds are enriched using LLM-guided splicing with PHP-native constructs from curated sources (I2). These seeds are then passed to the translation module's execution and repair loop for metric collection.

These components operate in a feedback-driven loop where execution outcomes inform scheduling priorities, guide mutations, and trigger repair routines.

4.3 Module 1: Translation

The translation module converts JavaScript seeds into PHP and incorporates a repair loop to recover malformed outputs.

JavaScript to PHP Translation. TRANSFUZZ begins with a corpus of JavaScript PoCs, collected from public exploit databases, fuzzing outputs, and regression test suites. Each input is processed by an LLM to generate a syntactically valid PHP equivalent. The resulting PHP seeds retain key artifacts of their JavaScript origin, such as deeply nested control flow, unconventional function calls, and anomalous constants, mimicking the high-impact structure of real exploits.

Each translated seed is executed using an instrumented PHP interpreter. If the seed fails with a syntax or runtime error, it enters the repair stage, where an LLM attempts to recover a valid variant using interpreter diagnostics. If execution succeeds or results in a crash, TRANSFUZZ logs the corresponding behavior and coverage signal. Only syntactically valid PHP seeds, either directly translated or successfully repaired, are forwarded to later stages. Translation is a one-time operation per seed.

Listing 1 shows a JavaScript PoC seed that constructs two arrays and defines a side-effecting getter using `__defineGetter__()`. The getter conditionally sets `b.length = 0xffffffff` when triggered, and the call to `a.concat(b)` invokes it, producing an oversized array. Listing 2 presents the corresponding PHP translation generated by TRANSFUZZ. Because PHP lacks native getter support and PHP arrays have no `length` field, the translation introduces a class `__Get` with a custom `__get()` method to emulate the getter's effect. The translation preserves the original seed's control flow (getter-driven callback, conditional branch); the allocator-state effect is recovered later during splicing with PHP-native primitives such as `preg_replace_callback()` and `str_repeat()` with extreme sizes.

Execution and Monitoring. All created PHP programs are executed in a sandboxed environment using the instrumented target PHP engine with sanitizers enabled. Each execution is monitored for three outcomes:

- **Success:** The program executes without crashing or triggering a runtime error. Using the recorded coverage, it is then scored and added to the seed pool for future splicing.
- **Crash:** The sanitizer detects anomalous behavior. These cases are triaged and logged for later analysis and bug reporting. The crashing seed is retained with high priority to encourage variant generation.
- **Error:** The program encounters a semantic or syntactic error. These seeds are routed to the LLM-based repair pipeline.

Repair Loop. To improve valid seed yield, TRANSFUZZ attempts to repair seeds that fail during execution. Using error messages produced by the PHP interpreter, the repair module constructs LLM prompts that include both the faulty seed and the specific diagnostic output. The LLM is then tasked with generating a corrected version. If the repaired seed still fails, the process is repeated up to a maximum of five attempts. Once a repaired seed executes successfully, its execution metrics are passed to the mutation module where it is scored (§4.4) and reintroduced into the seed corpus. Seeds that exceed token length limits or repeatedly fail repair are discarded to conserve resources.

4.4 Module 2: Mutation

The mutation module enriches the corpus of translated seeds by splicing them with PHP-native auxiliary seeds. It uses performance metrics collected from the instrumented PHP interpreter to score each seed, guiding the scheduler in selecting high-impact pairs for splicing.

Auxiliary Seed Generation. To recover PHP-specific diversity lost during translation, TRANSFUZZ incorporates *auxiliary seeds* ($S_{Auxiliary}$), syntactically valid PHP snippets that introduce PHP-specific constructs. We generate auxiliary seeds through three mechanisms:

- **Grammar-based Generation:** We employ a PHP grammar tailored to produce extreme constants, nested expressions, and variadic function calls. See Appendix Listing 11 for an example.
- **LLM-guided API Probing:** We prompt the LLM to generate code that uses randomly selected functions from a database of 883 PHP built-in functions (collected via `get_defined_functions()`). An example auxiliary seed with `preg_replace_callback()` nested within `var_dump()` appears in Listing 3.
- **Official PHP Test Suite:** We extract over 20,000 test cases from the PHP regression suite [32]. These samples provide canonical usage patterns, edge-case behavior, and regression-triggering inputs grounded in real-world interpreter changes.

To improve system efficiency, translation and auxiliary seed generation are executed offline in our implementation, front-loading compute-intensive LLM calls so that the fuzzing loop focuses on mutation and execution. TRANSFUZZ also supports online seed generation for settings where the JavaScript corpus evolves over time.

Seed Scoring. While coverage is the most common metric used in determining seed quality [33], TRANSFUZZ evaluates seeds with three additional metrics tailored to this environment. Each valid PHP seed (from translation or splicing) is scored before being stored in the seed corpus ($S_{Trans\&Derived}$):

- **Basic Block Coverage (N_{cov}):** The number of unique basic blocks exercised during seed execution.
- **Crash Anomaly ($B_{anomaly}$):** A binary flag indicating whether the seed caused the interpreter to crash (1) or not (0). Crashing seeds guide the generation of adjacent inputs that may reveal similar bugs.
- **Seed Ancestry ($N_{ancestry}$):** The number of JavaScript-translated seeds in the given seed's ancestry. Higher values indicate greater cross-language influence.

Listing 1: JavaScript Seed Example

```
a = [1]; b = [];
a.__defineGetter__(0, function () {
    if ("b" in window) {
        b.length = 0xffffffff;}});
c = a.concat(b);
```

Listing 2: PHP seed translated from JavaScript in Listing 1

```
$a = array(1); $b = array();
class __Get {
    public function __construct($b) {
        $this->b = $b;}
    public function __get($name) {
        if ($name == 'b') {
            $this->b = array();
            $this->b[] = 0xffffffff;}
        return $this->b;}}
$b = new __Get($b);
$c = array_merge($a, $b->__get('b'));
```

Listing 3: Auxiliary PHP seed

```
var_dump(preg_replace_callback('|(?P<name>)(\d+)|','
    callback', 'bh'));
```

Listing 4: The resulting seed from splicing Listing 3 and Listing 2

```
class __Get {
    public function __construct($b) {
        $this->b = $b;}
    public function __get($name) {
        if ($name == 'pattern') {
            $this->b ='|(?P<name>)(\d+)|';
        } elseif ($name =='match') {
            $this->b = array();
            $this->b[] = 0xffffffff;}
        return $this->b;}}
$b = new __Get('');
$pattern = $b->__get('pattern');
preg_match($pattern,$b->__get('match')[0],$m);
var_dump($m);
```

- **Token Length (N_{token}):** Used to penalize longer seeds that risk exceeding model token budgets.

Scores N_{cov} and $N_{ancestry}$ are rescaled using min-max normalization across the current batch to N'_{cov} and $N'_{ancestry}$; $B_{anomaly}$ is already binary ($\in [0,1]$) and is not rescaled. A token-length penalty is defined as:

$$penalty = 1 - \frac{\min(N_{token}, Max_{token})}{Max_{token}} \tag{2}$$

We then compute a composite score as follows:

$$Score = penalty \cdot (w_1 \cdot N'_{cov} + w_2 \cdot B_{anomaly} + w_3 \cdot N'_{ancestry}) \tag{3}$$

We assign equal weights $w_1 = w_2 = w_3 = 1/3$. We set $Max_{token} = 5{,}000$, conservatively below the 8,000-token limit of `Meta-Llama-3-8B-Instruct`; prior work shows output quality degrades as prompts approach model limits [34]. Seeds at or above Max_{token} receive a score of zero.

In Listing 2, the translated seed covers 4,647 basic blocks, has no anomaly ($B_{anomaly} = 0$), has $N_{ancestry} = 1$, and $N_{token} = 110$. After min-max normalization, $N'_{cov} = 0.90$ and $N'_{ancestry} = 1$, and the token penalty is 0.99. The resulting composite score is 0.63.

Seed Splicing and Pair Scheduling. To preserve the anomalous structures and control flows of the original seed while amplifying it with interpreter-specific features, TRANSFUZZ generates new seeds by splicing JavaScript-derived seeds with auxiliary PHP seeds. Each pair is passed to an LLM to produce a new composite program.

Energy Assignment. Equation 4 computes the energy assigned to each seed. s denotes a JavaScript-translated or derived seed and $Score_s$ is from Equation 3. Energy is derived through scaled normalization and rounded up; we set $E_{max} = 5$ as the maximum number of pairs per seed.

$$\texttt{GetEnergy}(s) = \lceil E_{\max} \cdot \frac{Score_s - Score_{\min}}{Score_{\max} - Score_{\min}} \rceil \tag{4}$$

Pair Scheduling Algorithm. Algorithm 1 outlines the pairing process. For each seed $s \in S_{\text{Trans\&Derived}}$, we use its assigned energy (Equation 4) to generate up to $E_{\max}$ pairings. Each pair selects a second seed by deterministic alternation on energy parity between the translated set ($S_{\text{Trans\&Derived}}$) and the auxiliary set ($S_{\text{Auxiliary}}$). `RandomPick` samples uniformly without replacement from the given set. After all seeds in $S_{\text{Trans\&Derived}}$ have been enumerated, the set is reset.

Seed Splicing. If the resulting seed executes successfully, it is added to the corpus and scored. If execution fails, the seed is routed through the LLM-based repair pipeline. Successfully repaired seeds are returned to $S_{\text{Trans\&Derived}}$. Listing 4 illustrates the output of splicing the translated seed of Listing 2 with an auxiliary seed (Listing 3): the spliced result maintains the getter-class structure while embedding regular-expression constructs from the auxiliary, ultimately triggering a memory leak (Bug #26).

5 Implementation

TRANSFUZZ is implemented in approximately 3,000 lines of Python. To capture coverage, we use a custom LLVM pass compiled into the PHP interpreter. When instrumented with `clang's -fsanitize-coverage=trace-pc-guard` option, this pass records basic block coverage information in a shared bitmap.

Given that the PHP interpreter exposes approximately 250K unique basic blocks, we allocate a 36KB (288K-bit) bitmap for efficient tracking. To detect memory and logic bugs, we enable `clang`'s AddressSanitizer (ASan) and UndefinedBe-

Algorithm 1: Seed Pairing Algorithm

Input: $S_{\text{Trans\&Derived}}$: The Set of Seeds Translated or Derived From JavaScript; $S_{\text{Auxiliary}}$: The Set of Auxiliary Seeds
Output: `Pairs`: The Produced Seed Pairs

Function `Pairing`($S_{Trans\&Derived}$, $S_{Auxiliary}$):
  $Pairs \leftarrow \{\}$
  foreach $s \in S_{Trans\&Derived}$ **do**
    $E \leftarrow$ `GetEnergy`(s)
    while $E \neq 0$ **do**
      if $E\%2 == 0$ **then**
        $s_T \leftarrow$ `RandomPick`($S_{\text{Trans\&Derived}}$)
        $Pairs \leftarrow Pairs \cup \{(s, s_T)\}$
      else
        $s_A \leftarrow$ `RandomPick`($S_{\text{Auxiliary}}$)
        $Pairs \leftarrow Pairs \cup \{(s, s_A)\}$
      end
      $E \leftarrow E - 1$
    end
  end
  $S_{\text{Trans\&Derived}} \leftarrow \{\}$
  return *Pairs*

haviorSanitizer (UBSan), both of which provide precise fault localization for interpreter-level errors.

Because TRANSFUZZ uses LLM interactions for translation, mutation, and repair, minimizing query latency is essential for system throughput. To this end, we deploy a local LLM instance and avoid reliance on external APIs. Due to resource limits, our implementation currently uses the Hugging Face version of `Meta-Llama-3-8B-Instruct` [35], an 8-billion parameter model offering a balance of capability, efficiency, and responsiveness on consumer-grade GPUs. For grammar-based PHP code generation, we adapt Domato [36] by pruning deprecated constructs from its grammar.

LLM Generation Parameters. Sampling parameters are tuned empirically per task. Translation and splicing use temperature 1.0, and repair uses temperature 0.3 (conservative edits guided by the error message). All tasks share top-$p = 0.95$, top-$k = 50$, and `max_new_tokens` $= 2048$.

GPU Deployment and Inference Throughput. Llama-3-8B fits within two NVIDIA Tesla P100 GPUs. We run independent replicas of Llama-3-8B on two GPUs (4 instances). Each replica serves translation, splicing, and repair requests for the fuzzing orchestrator, which load-balances across the four replicas.

6 Evaluation

We evaluate TRANSFUZZ to answer the following research questions:

- **RQ1:** Can TRANSFUZZ discover new, unknown bugs in the PHP interpreter?
- **RQ2:** How does each technical innovation impact the overall performance of TRANSFUZZ? How does each auxiliary seed type contribute to overall performance?
- **RQ3:** How does TRANSFUZZ compare to state-of-the-art fuzzers/frontier LLM coding agents?

Table 2: Bugs confirmed by the PHP development team.

#	Ver.	Issue ID	Class	Comp.	Status	Fix
1	8.3.9	[redacted]	CWE-190	Date	Dp	
2	8.3.9	[redacted]	CWE-190	Standard	Dp	
3	8.3.9	[redacted]	CWE-190	Date	Vd	
4‡P1	8.3.9	[redacted]	CWE-401	DOM	Vd	
5	8.3.9	[redacted]	CWE-190	Date	Dp	
6	8.3.9	[redacted]	CWE-190	Engine	Fx	+17 -1
7	8.3.9	[redacted]	CWE-190	Date	Fx	+18 -2
8	8.3.9	[redacted]	CWE-190	-	Vd	
9	8.3.9	[redacted]	CWE-190	-	Vd	
10	8.3.9	[redacted]	CWE-190	Session	Fx	+27 -3
11	8.3.9	[redacted]	CWE-190	Date	Vd	
12*‡P1	8.3.9	[redacted]	CWE-476	-	Dp	
13*‡P1	8.3.9	[redacted]	CWE-476	SimpleXML	Dp	
14	8.3.9	[redacted]	CWE-190	Date	Vd	
15*‡P1	8.3.9	[redacted]	CWE-188	Engine	Dp	
16*‡P1	8.3.9	[redacted]	CWE-401	DOM	Vd	
17	8.3.9	[redacted]	CWE-190	Date	Vd	
18*‡P1	8.3.9	[redacted]	CWE-188	-	Dp	
19*†P1	8.3.9	[redacted]	CWE-416	Engine	Vd	
20	8.4.1	[redacted]	CWE-190	Standard	Dp	
21	8.4.1	[redacted]	CWE-190	Date	Vd	
22	8.4.1	[redacted]	CWE-190	-	Vd	
23	8.4.1	[redacted]	CWE-190	-	Vd	
24	8.4.1	[redacted]	CWE-561	Streams	Vd	
25	8.4.1	[redacted]	CWE-190	Date	Vd	
26*‡P1	8.4.1	[redacted]	CWE-401	PCRE	Fx	+73 -33
27*	8.4.1	[redacted]	CWE-190	Date	Vd	
28*†P1	8.4.1	[redacted]	CWE-416	Streams	Fx	+19 -4
29	8.4.1	[redacted]	CWE-190	Date	Vd	
30	8.4.1	[redacted]	CWE-190	-	Fx	+18 -2
31†	8.4.1	[redacted]	CWE-416	Streams	Fx	+23 -8
32	8.4.1	[redacted]	CWE-190	Date	Dp	
33*‡P1	8.4.1	[redacted]	CWE-416	Engine	Dp	
34	8.4.1	[redacted]	CWE-190	Date	Vd	
35‡P1	8.4.1	[redacted]	CWE-401	libXML	Vd	
36	8.4.1	[redacted]	CWE-190	Date	Vd	
37	8.4.1	[redacted]	CWE-190	-	Vd	
38*	8.4.1	[redacted]	CWE-561	Engine	Fx	+27 -3
39	8.4.1	[redacted]	CWE-190	Date	Vd	
40*	8.4.1	[redacted]	CWE-190	-	Vd	
41*	8.5.0	[redacted]	CWE-617	Streams	Fx	+39 -1
42*‡P1	8.5.0	[redacted]	CWE-401	pdo_firebird	Vd	
43*†P1	8.5.0	[redacted]	CWE-416	spl	Fx	+24 -2
44*‡P1	8.5.0	[redacted]	CWE-401	readline	Fx	+44 -22
45*	8.5.0	[redacted]	CWE-561	Streams	Fx	+20 -1
46*‡P1	8.5.0	[redacted]	CWE-401	dba	Fx	+28 -12
47*	8.5.0	[redacted]	CWE-190	filter	Fx	+15 -1
48*	8.5.0	[redacted]	CWE-190	filter	Dp	
49*	8.5.0	[redacted]	CWE-190	spl	Fx	+22 -1
50*	8.5.0	[redacted]	CWE-190	Date	Fx	+32 -10

† RCE primitive. ‡ DoS primitive. * Required JS structure for trigger. P1-3 reachability as defined by §3.

- **RQ4:** How effective is TRANSFUZZ overall at generating semantically valid and executable PHP seeds?
- **RQ5:** How does model choice affect performance?

6.1 Experimental Setup

We evaluate TRANSFUZZ on `PHP 8.3.9`, `8.4.1`, and `8.5.0-dev` using a cluster with 8 NVIDIA Tesla P100 GPUs. Our source corpus contains 605 JavaScript PoCs from three public repositories.[1] Translation produces 456 valid PHP seeds. We additionally prepare 8,500 auxiliary seeds, evenly drawn from our grammar generator, LLM-generated PHP

[1] https://github.com/rycbar77/V8Exploits, https://github.com/drtychai/browser-exploitation, https://github.com/allpaca/V8Harvest/blob/master/V8Harvest_2020.md

snippets, and the official PHP test suite [32]. Unless otherwise noted, experiments targeting PHP `8.4.1` use seven independent 24-hour runs.

Common Protocol. We deduplicate bugs by sanitizer stack trace and manually triage each finding. Baseline experiments use approximately matched daily compute budgets (TRANSFUZZ: $13.44; baselines: $13.20; frontier agent: $13.50). Following fuzzing-evaluation best practices [37, 38], all 24-hour ablation and comparison experiments are repeated seven times, and significance is assessed using Mann–Whitney U tests ($p < 0.05$).

6.2 Results

6.2.1 Vulnerability and Bug Discovery

To assess the bug finding capabilities of TRANSFUZZ, for three PHP versions, we run a 14-day campaign with ASan, UBSan, and our LLVM coverage instrumentation. We manually triage each finding to assess its security relevance.

Table 2 lists the set of confirmed bugs. The *Version* column indicates the affected version, the *Class* column lists the corresponding Common Weakness Enumeration (CWE), and the *Component* column specifies the impacted subsystem. The *Status* column indicates if the bug is fixed (*Fx*), verified (*Vd*), or a duplicate (*Dp*), and the *Fix* column reports the number of lines changed. Bugs are marked with their vulnerability primitive and reachability as defined by §3 and if they involved JavaScript structure in their trigger.

Over a 14-day fuzzing campaign per target, TRANSFUZZ discovered **50 bugs** across three PHP versions: `PHP 8.3.9`, `PHP 8.4.1`, and `PHP 8.5.0-dev`. Of these, **39 were previously unknown** and have been acknowledged by the PHP development team, and **16 are security relevant memory-safety violations** under our threat model. To assess the contribution of JavaScript-transferred structures, we manually minimized the triggering inputs, removing JavaScript-derived structures until the bug no longer reproduced. This analysis found that **22 bugs relied on JavaScript-derived structures in their triggers**. All findings were responsibly disclosed via the official PHP security reporting process.

Vulnerability Distribution. The discovered bugs correspond to **7 distinct CWEs** [2] and affect a broad cross-section of the interpreter, spanning core subsystems (`Engine`, `Date`, `Streams`), parsing libraries (`DOM`, `libXML`), and extensions (`spl`, `filter`). The primary vulnerability classes were:

(1) Integer overflows. Unsanitized input in core date and time modules triggers signed integer overflows that produce undefined behavior. Although these do not immediately crash the interpreter, integer overflows can be chained to secondary vulnerabilities such as buffer overflows. For instance, CVE-2024-11236 chains an integer overflow in `ldap_escape()` into an out-of-bounds write [29]. We conservatively classify non-crashing integer overflows as non-security relevant under our analysis scope (§3), but note that those in modules accepting user input (e.g., `Date`) warrant further investigation via P3.

(2) Use-after-frees (UAFs). Five UAFs were discovered in memory management paths such as `zend_objects_API.h` and `user_filters.c`. UAFs are high-severity primitives. In the PHP interpreter context, UAFs have enabled RCE (e.g., CVE-2024-11235 [1]).

(3) Memory leaks. Seven bugs cause unbounded memory growth. Under sustained invocation, realistic in long-running PHP-FPM workers or shared hosting, these leaks degrade performance and can lead to DoS through resource exhaustion.

(4) Null/misaligned pointer dereferences and unreachable paths. Four bugs involve invalid memory accesses (null dereferences, misaligned reads), each producing a segmentation fault. Additional bugs reach program points marked unreachable by developers, exposing control flow inconsistencies.

Security Relevance. Under the deployment model of §3, these bugs expose security-relevant interpreter primitives under P1. Memory leaks can enable resource-exhaustion attacks, invalid memory accesses can terminate long-running PHP workers and cause DoS, and use-after-free conditions constitute memory-corruption primitives that, depending on the deployment environment and available mitigations, may enable RCE. Prior PHP interpreter vulnerabilities demonstrate that such memory-corruption primitives can be developed into code-execution or sandbox-escape exploits [2, 30]. TRANSFUZZ does not perform this exploit-development.

> TRANSFUZZ discovers 50 bugs across three PHP versions, including 16 security-relevant memory-safety vulnerabilities. 39 were previously unknown.

6.2.2 Ablation Study

We evaluate four variants removing one component at a time: **No Translation**, **No Splicing**, **No Scheduling Algorithm** (uniform random scheduling), and **No Repair**. We separately evaluate the auxiliary-seed sources by enabling only the PHP test suite, LLM-generated seeds, or grammar-generated seeds and by disabling all auxiliary-seed sources. All results in Table 3 are statistically significant.

RQ2.1. Key Technical Innovation Impact. Figure 3 reports the average (arithmetic mean) coverage trend across the 24-hour fuzzing campaign for each ablated system variant. The results demonstrate that all tested components positively contribute to overall system performance. The complete TRANSFUZZ achieves a peak of 41.7K unique basic blocks covered, substantially outperforming any ablated variant. Removing cross-language seed translation (I1) reduces coverage

[2] The bugs found by TRANSFUZZ correspond to the following CWEs: CWE-190 Integer Overflow or Wraparound, CWE-401 Missing Release of Memory, CWE-476 Null Pointer Dereference, CWE-188 Reliance on Data Layout, CWE-416 Use-after-Free, CWE-561 Unreachable Program Point, CWE-617 Reachable Assertion. [39]

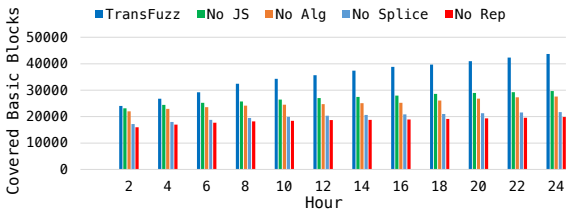


Figure 3: Coverage with Ablated Components

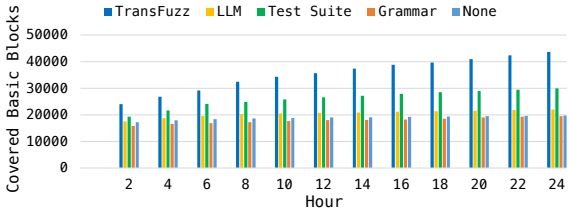


Figure 4: Coverage with Ablated Auxiliary Seeds

to 29.7K; disabling splicing (I2) results in 21.7K; omitting ancestry-aware scheduling (I3) drops coverage to 27.6K; and disabling seed repair (I4) leads to the lowest performance at 19.8K. These results highlight that each component addresses a specific bottleneck in the pipeline, either by improving the quality of seeds or by enhancing system efficiency and execution robustness:

Cross-language seed translation (I1) provides a structurally diverse and semantically rich initial corpus. JavaScript PoCs often contain deeply nested control and data flows that are difficult to generate from scratch. These patterns enhance early-stage interpreter exploration and bootstrap the corpus with productive seeds. The significant drop in coverage when this component is removed indicates that auxiliary seeds alone are insufficient to explore deep behaviors. **Seed splicing (I2)** increases diversity by embedding translated seeds with PHP-native constructs. This facilitates the emergence of rare execution paths by introducing language-specific idioms and corner cases. The notable decline in system performance caused by the removal of splicing suggests that semantic enrichment via splicing is critical for sustained fuzzing progress beyond the initial seed set. **Ancestry-aware scheduling (I3)** ensures that fuzzing effort is directed toward seeds with high payoff potential, balancing coverage exploration and crash reproduction. Random scheduling, used in the ablation, leads to over-sampling of unproductive seeds, stalling feedback loops. This demonstrates the importance of prioritization strategies tailored to multi-source seed corpora. **Seed repair (I4)** addresses a key limitation of LLM-based generation: the tendency to produce malformed or syntactically invalid code. Without repair, these inputs are discarded, reducing the effective throughput of the system. The low coverage in the no-repair configuration emphasizes the importance of

Ablated Variant	Coverage	P-value	Bugs	P-value
No Scheduling	27,576	0.0006	2.29	0.0098
No Splicing	21,700	0.0006	1.14	0.0019
No Repair	19,849	0.0006	2.01	0.0034
No Translation	29,659	0.0006	1.14	0.0019
Test Suite Only	29,991	0.0006	2.43	0.0089
LLM-generated Only	21,990	0.0006	2.37	0.0438
Grammar-generated Only	19,541	0.0006	1.42	0.0024
No Aux. Seeds	18,716	0.0006	1.28	0.0019
TRANSFUZZ	**41,697**	-	**4.71**	-

Table 3: P-values of ablation study

recovery mechanisms in LLM-integrated fuzzing pipelines, especially when working with long or spliced inputs.

Taken together, the results suggest that the components are synergistic: the benefits of cross-language seeding (I1) and splicing (I2) rely on robust execution (I4) and informed selection (I3). This highlights the need for TRANSFUZZ's holistic design, where each module supports and amplifies the other to achieve high coverage and bug discovery rates.

RQ2.2. Auxiliary Seed Type Impact. Figure 4 presents the coverage trends from our second ablation study, which isolates the individual contributions of each auxiliary seed type. The full configuration achieves the highest peak coverage at 41.7K basic blocks. When restricted to only one seed type, performance degrades. The PHP test suite seeds yield 30.0K, LLM-generated seeds achieve 22.0K, and grammar-generated seeds reach 19.5K. Finally, removing all auxiliary seeds yielded the worst performance at 18.7K.

These results highlight the importance of auxiliary seed diversity. Each auxiliary source introduces unique traits. Grammar-generated seeds emphasize edge-case constants and extreme syntax. LLM-generated seeds provide creative function call usage. The PHP regression test suite offers a high-quality corpus of inputs that systematically exercise interpreter behavior. Their thorough and specification-driven nature contributes significantly to broad structural and semantic coverage.

Each key innovation contributes to the overall performance of TRANSFUZZ. By contributing unique code attributes, each auxiliary seed type enhances the quality of generated seeds.

6.2.3 Baseline Comparison Study

We compare against three representative PHP-capable fuzzers: **Domato** [8], **PolyGlot** [2, 40], and **FlowFusion** [10, 41]. For a controlled comparison, we execute against the same instrumented PHP `8.4.1` build with the same initial corpus for each fuzzer (PHP test suite with translated seeds). We measure basic-block coverage and unique bugs. We further compare TRANSFUZZ with **Claude Code (Opus 4.8)**. The agent receives the PHP source, test suite, and sanitizer-instrumented

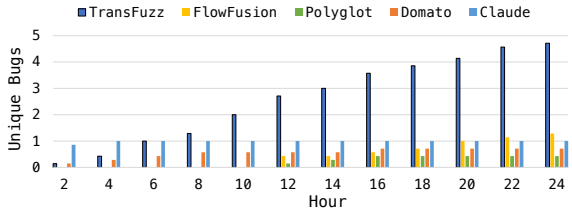


Figure 5: Bug Comparison against State-of-the-Art

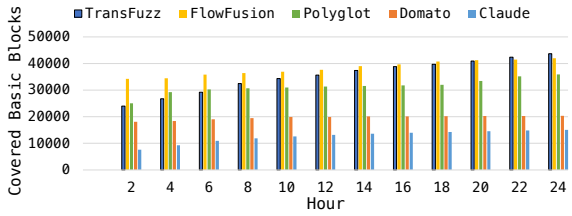


Figure 6: Coverage Comparison against State-of-the-Art

Listing 5: The PHP PoC for Bug #19

```php
preg_match('/./', chr(PHP_INT_MAX));
class Test implements \IteratorAggregate{
    public function getIterator(): \Generator{
        yield 'foo';
        while (true) {
            Fiber::suspend();
            $o = array();
            $g = function() use (&$o) {
                $o['x'] = 1;
                return array_merge($o,array());
                };
            gc_collect_cycles();
            $o['x'] = PHP_INT_MAX;
            $o['y'] = PHP_FLOAT_MIN;
            $o['z'] = "Infinity";
            call_user_func($g); } } }
function f() {yield from new Test();}
$iterable = f();
$loop = new Fiber(function() use (&$callback){
    $callback->start();});
$callback = new Fiber(function () use ($iterable) {
    while (true) {$iterable->next();}});
$loop->start();
```

build, but no network access, known-bug information, or target files. We perform seven 24-hour trials and measure coverage and bug discovery under a matched compute budget.

RQ3.1. State-of-the-Art Fuzzers. To assess the performance of TRANSFUZZ relative to state-of-the-art systems, we compare the 24-hour bug discovery and coverage trends of FlowFusion, PolyGlot, Domato, and TRANSFUZZ (Figure 5 and Figure 6). While TRANSFUZZ averaged the highest basic block coverage (**41.7K**), the delta between TRANSFUZZ and FlowFusion (41.5K) was not statistically significant ($p > 0.05$). However, TRANSFUZZ demonstrated a dramatic leap in bug-finding efficacy, averaging 4.71 unique bugs per run; a 3.7× improvement over FlowFusion (1.28), a 4.7× improvement over Claude Code (1.00), and over 6× that of Domato (0.71) and PolyGlot (0.43). All bug discovery improvements were statistically significant (Table 4).

These results highlight a critical "coverage-discovery" gap in existing PHP fuzzers. Domato is fundamentally restricted by the simplicity of its grammar, and PolyGlot lacks PHP-specific mutational power. FlowFusion attains coverage comparable to TRANSFUZZ, yet finds substantially fewer bugs which points to two structural limitations. First, FlowFusion's data-flow mutations lack mechanisms to synthesize control-flow constructs. This is consequential because 96% of the PHP test suite is control-flow-free [10], thus seeds inherit a flat structure. Second, FlowFusion operates without a feedback loop. Because of this, when FlowFusion generates a valuable seed (e.g., crashing or new-coverage discovering), the fuzzer has no signal to prioritize or build upon it.

Baseline	Coverage	P-value	Bugs	P-value
FlowFusion	41,521	0.7974	1.28	0.0019
PolyGlot	28,216	0.0006	0.43	0.0018
Domato	20,090	0.0006	0.71	0.0016
Claude Code	15,184	0.0006	1.00	0.0016
TRANSFUZZ	**41,697**	-	**4.71**	-

Table 4: P-values of comparison study

In contrast, TRANSFUZZ actively generates loops and conditionals and uses coverage feedback to refine generation. The result is higher bug density (bugs found per covered block) at comparable coverage. Furthermore, because TRANSFUZZ actively introduces new constructs, rather than combining existing structures like FlowFusion, TRANSFUZZ can discover bugs unreachable by other systems.

RQ3.2. Frontier Model. Over seven independent 24-hour trials, Claude Code fell short of TRANSFUZZ. The agent averaged 15.2K basic blocks and 1.00 bugs. Despite being given a sanitizer-instrumented PHP interpreter, the agent primarily produced false positive memory failures. PHP contains the Foreign Function Interface (FFI) module which allows for C-style memory manipulation. The agent would target this module to manually trigger the sanitizer. Per the developers, "*Using the FFI extension... allows you to do anything C can do, including manual memory management.*" This result highlights an important distinction between general-purpose code agents and security-oriented fuzzing systems: *a more capable model alone does not guarantee effective vulnerability discovery*. Without the corpus, feedback loop, and orchestration provided by TRANSFUZZ, the agent struggled to discover meaningful interpreter vulnerabilities.

Case Study: Bug #19 (Fiber UAF). To illustrate mechanistically why transferred scaffolding unlocks a bug class, we walk through Bug #19, a use-after-free in `zend_objects_API.h`. The original V8 PoC (CR-786723, Appendix Listing 12) nests a closure that mutates an outer-scope object, with an explicit `gc()` and `%OptimizeFunctionOnNextCall`.

Translation preserved the nested-closure structure: `$o` is captured by reference into `$g`, invoked via `call_user_func`, and `gc()` becomes `gc_collect_cycles()`. While the translated seed alone does not trigger a UAF, splicing with PHP-native auxiliaries introduced two further primitives: `Fiber::suspend()` and a `Generator` body driven by `yield` from an `IteratorAggregate` test case. In the final PoC (Listing 5), a generator yields inside a `while(true)` loop that invokes `Fiber::suspend()`; an outer Fiber holds the generator, and on resumption the closure `$g` references an array `$o` that the parent context has released, dereferencing freed memory. Three primitives had to coincide: generator iteration, Fiber suspension, and closure-by-reference. Dataflow mutation alone does not introduce any of the three, and no grammar-based generator would sample the specific three-way nesting by chance; all three arrived through the translation plus splicing pipeline.

TRANSFUZZ achieves coverage comparable to state-of-the-art fuzzers, while significantly outperforming in bug discovery. This is driven by TRANSFUZZ's ability to create complex program structures.

6.2.4 System Performance

We compute the structural preservation score (SP, Equation 1) for every JavaScript/PHP pair and measure system throughput using the number of LLM prompts, valid PHP seeds, and repair attempts during a 24-hour run.

Over the corpus of JavaScript seeds and their PHP translations, we observe an average SP of **77%**. This high correspondence suggests that the translation oracle preserves most structures needed for exercising interpreter logic. The discrepancies primarily stem from API-level differences and environment-level constructs (e.g., compiler options and language-specific structures). Overall, these results indicate that the translations adequately preserve semantic fidelity, maintaining the structural features necessary for effective cross-language fuzzing.

Over 24 hours, TRANSFUZZ issued a total of 15,260 prompts, averaging **635 prompts/hour**. During the same period, it successfully generated **3,193 valid PHP scripts** (6,740 total generated), resulting in an average yield of **approximately 4.8 prompts per valid seed**. TRANSFUZZ achieves this throughput through deliberate choices that maximize LLM utility. TRANSFUZZ's use of rich context, strict enforcement of context window limits, and highly constrained, close-ended queries enables small models to reliably perform translation, splicing, and repair without sacrificing output quality.

After 24 hours, 32% of the scripts are error-free; 8% are repaired after one prompt; 3% after two prompts; 2% after three prompts; and 1% each after four and five prompts. However, 53% of the generated scripts remain irreparable even after all five attempts. The process of splicing increases both the size and complexity of the resulting seeds. Naturally, this stochastic process tends to introduce more syntactic and semantic errors into a seed, thus resulting in a seed which requires more than five fix attempts. Notably, auxiliary seeds derived from the PHP test suite often rely on external files (e.g., .txt or .inc) or compilation configurations. In such cases, the dependencies fall outside the scope of the repair pipeline and cannot be resolved automatically. To prevent the repair loop from stalling, we discard seeds after five fix attempts.

Due to LLM inference, TRANSFUZZ is slower than baselines. Over 24 hours, PolyGlot, FlowFusion, and Domato respectively averaged throughputs of 1858K, 47.3K, and 24.7K seeds. However, because of the repair loop, TRANSFUZZ produces the highest proportion of valid seeds (47%) compared to PolyGlot (39.7%), FlowFusion (2.5%), and Domato (1.9%).

Parameter Choices. Hyperparameters are set from first principles and pilot runs; a comprehensive sweep is future work. $E_{\max} = 5$ balances exploration against batch turnover. $Max_{\text{token}} = 5{,}000$ is 62.5% of Llama-3-8B's 8K context. A value conservatively set to avoid significant model quality degradation [34]. 50/50 translated/auxiliary pairing balances cross-language influence against PHP-native enrichment. Equal weights $w_1 = w_2 = w_3 = 1/3$ preserve generalization. Five-attempt repair cutoff is justified by the yield curve falling below the compute-cost floor by attempt 6.

The translation oracle retains 77% of source structure; TRANSFUZZ produces 3,193 valid PHP seeds per 24 hours, enabled by the repair loop and parameter choices.

6.2.5 Model Comparison.

To analyze whether the finding depends on a specific LLM, we re-run TRANSFUZZ with three comparable, instruction-tuned models: `Meta-Llama-3-8B-Instruct`, `Mistral-7B-Instruct`, and `Qwen2-7B-Instruct`. Each variant runs for seven independent 24-hour trials against PHP `8.4.1`. Over seven 24-hour trials, the three models differ minimally (Table 5): Llama-3-8B-Instruct (41.7K BBs, 4.71 bugs), Qwen2-7B-Instruct (40.5K BBs, 3.98 bugs), Mistral-7B-Instruct (40.2K BBs, 3.82 bugs); Mann-Whitney U reveals the differences are not significant.

Model	Coverage	P-value	Bugs	P-value
Qwen	40,531	0.7974	3.98	0.1921
Mistral	40,213	0.8316	3.82	0.2834
Llama	41,697	-	4.71	-

Table 5: Model comparison results.

Within 7–8B open-weight instruction-tuned LLM class, model choice does not meaningfully affect TRANSFUZZ's performance.

7 Limitations and Future Work

Repair limitations. Seed repair is a critical component of TRANSFUZZ, as it enables the recovery of otherwise unusable seeds. In the current implementation, each seed is allotted up to five repair attempts before being discarded. In our evaluation, 53% of seeds remain irreparable. As previously noted, auxiliary seeds derived from the PHP regression test suite have external dependencies or nonstandard compilation configurations. These dependency issues fall outside the scope of automated program repair techniques [42–44] as they require external environment modifications.

Memory Leaks and Multi-Tenant Deployments. Seven of the security relevant vulnerabilities are memory leaks. Under a strict same-principal threat model, a leak triggered by the attacker's own code is self-DoS. The realistic deployment surface is multi-tenant. Long-running PHP-FPM workers in shared hosting and plugin ecosystems (WordPress, Drupal, Magento) execute third-party code with platform privileges, so a leak in one tenant or plugin degrades others via shared-allocator state, worker OOM kills, and pool exhaustion. We classify these as security relevant under the multi-tenant model but note they do not count under a single-tenant scope.

8 Related Work

Fuzzing Language Interpreters. Fuzzing has proven highly effective for uncovering software vulnerabilities across diverse domains [45], including compilers [46, 47], hardware [48], operating systems [49, 50], and deep learning libraries [51]. In the domain of language interpreters, JavaScript engines such as V8 have been extensively fuzzed using grammar-based generators [8], structural mutation tools [52], and deep learning techniques [53, 54].

JavaScript fuzzers implement a variety of complex fuzzing strategies ranging from grammar-based fuzzing to intermediate representation mutations to token-based fuzzing [55]. Grammar-driven JavaScript fuzzers include Domato [8], which generates programs from a hand-crafted grammar, and LangFuzz [52], which recombines real-world JS programs to form semantically valid inputs. CodeAlchemist [56] uses program analysis to construct semantically constrained JS code. DIE implements aspect-preserving mutation to preserve valuable properties across mutations [57]. More recent work [5, 54] introduces intermediate representations like FuzzIL and FlowIR to guide structured mutation. Neural systems such as Montage [53, 58] learn patterns in existing corpora to synthesize new JavaScript inputs.

In contrast, PHP has received comparatively little fuzzing attention. Tools like NAUTILUS [6] and Gramatron [7] use manually constructed grammars to generate fuzzing inputs. PolyGlot [2] uses a uniform IR to generate programs across multiple languages. Most recently, FlowFusion [10] mutates PHP test cases using dataflow analysis, interface fuzzing, and environment crossover.

Paper	Year	Generative	Cross-lang	Feedback Loop	LLM Usage	Splicing
Domato [8]	2017	✓				
NAUTILUS [6]	2019	✓		✓		
Gramatron [7]	2021	✓		✓		
PolyGlot [2]	2021	✓	✓			
FlowFusion [10]	2024					✓
TRANSFUZZ	2026	✓	✓	✓	✓	✓

Table 6: Comparison with existing PHP fuzzers.

TRANSFUZZ takes a fundamentally different approach to fuzzing PHP (Table 6): it reuses JavaScript PoCs by translating them into PHP via large language models. Unlike FlowFusion, which mutates known seeds through data-flow–driven edits, TRANSFUZZ injects diverse and complex control-flow structures derived from JS exploits. It further integrates LLM-guided splicing and adaptive prioritization to drive a feedback-aware fuzzing loop.

LLMs for Program Generation and Fuzzing. LLMs are increasingly used to generate test inputs [59–61], fuzzing harnesses [62], and mutated seeds [63, 64]. CODAMOSA [60] uses coverage feedback to prompt Codex for new unit tests. FuzzGPT [63] and TitanFuzz [65] generate seeds for deep learning libraries using GPT-style models. Fuzz4All [64] combines GPT-4 and StarCoder to mutate and filter code snippets.

TRANSFUZZ extends this line of work along a couple of novel dimensions. First, it applies LLMs to cross-language seed translation, a less explored setting where exploit semantics must be preserved across runtime environments. Second, it fuses LLM-translated inputs with native PHP snippets using LLM-guided splicing, creating hybrid seeds that retain high structural and semantic diversity. This pipeline enables TRANSFUZZ to produce diverse and effective seeds without requiring full grammar definitions or corpus bootstrapping.

9 Conclusion

We presented TRANSFUZZ, a cross-language fuzzing system that transfers exploit-derived execution scaffolding from JavaScript to PHP. Our results show that this transferred structure remains useful across the language boundary. TRANSFUZZ achieves coverage comparable to state-of-the-art PHP fuzzing while discovering substantially more bugs. Across three PHP versions (`8.3.9`, `8.4.1`, and `8.5.0-dev`), TRANSFUZZ uncovered 50 bugs, including 39 previously unknown and 16 security-relevant vulnerabilities. These results demonstrate that exploit corpora from mature ecosystems can provide valuable seeds for testing under-studied interpreters.

Ethical Considerations

Stakeholders. The following stakeholders were considered during the development and testing of TRANSFUZZ: (1) *PHP interpreter developers*: who are responsible for the creation and maintenance of the official PHP interpreter, (2) *PHP application developers*: who use PHP for application development, (3) *end users*: who rely on PHP-powered services, and (4) *security researchers*: who develop tools for automated program testing and verification.

Impacts. This work benefits the PHP ecosystem and its users, enabling PHP developers to find and patch bugs before end users are impacted. It provides a new state-of-the-art tool to better assess both the security and functionality of the official PHP interpreter.

Mitigations. To prevent ethical risks, we took the following steps: (1) all experiments were conducted locally within isolated, sandboxed environments with proper permissions and (2) all discovered vulnerabilities were responsibly disclosed to the PHP development team in accordance with their established disclosure procedure. Moreover, we acknowledge potential ethical concerns associated with leveraging LLMs for generating exploit-like code. To mitigate misuse, we emphasize the responsible dissemination of TRANSFUZZ and its results.

Decision. We carefully considered the ethical implications of this work through every step of development and deployment. We conclude that the benefits of this research (enhanced PHP security, new state-of-the-art security techniques) outweigh the mitigated risks. Thus, the conducting and publishing of this research is ethically justified and in the public interest.

Open Science

We provide the source code of TRANSFUZZ, available at the anonymous link[3]. The repository includes:

- An overview of the structure of TRANSFUZZ and the instructions on the operation of TRANSFUZZ.
- **gen_0**: A corpus of translated JavaScript exploit PoCs.
- **request_handler.py**: A Python script to generate seeds.
- **execution_handler.py**: A Python script to execute seeds.
- **similarity_counter**: Structural preservation score calculator.

Along with additional scripts for LLM interactions, coverage calculation, sanitization, and seed scoring. By providing these resources, we aim to enable the community to validate our findings, extend TRANSFUZZ, and apply our methodology.

[3] https://anonymous.4open.science/r/transfuzz-7723723/README.md

References

[1] CVE Details. Cve-2024-11235. https://www.cvedetails.com/cve/CVE-2024-11235/, 2024.

[2] Yongheng Chen, Rui Zhong, Hong Hu, Hangfan Zhang, Yupeng Yang, Dinghao Wu, and Wenke Lee. One engine to fuzz'em all: Generic language processor testing with semantic validation. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 642–658. IEEE, 2021.

[3] SSD Secure Disclosure. Vulnerability publication - SSD disclosure. https://ssd-disclosure.com/category/vulnerability-publication/, 2024.

[4] CVE Details. Cve-2025-6491. https://www.cvedetails.com/cve/CVE-2025-6491/, 2025.

[5] Samuel Groß, Simon Koch, Lukas Bernhard, Thorsten Holz, and Martin Johns. Fuzzilli: Fuzzing for javascript jit compiler vulnerabilities. In *NDSS*, 2023.

[6] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. Nautilus: Fishing for deep bugs with grammars. In *NDSS*, 2019.

[7] Prashast Srivastava and Mathias Payer. Gramatron: effective grammar-aware fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2021, page 244–256, New York, NY, USA, 2021. Association for Computing Machinery.

[8] Ivan Fratric. The great dom fuzz-off of 2017, 2017.

[9] Linkai Zheng, Xiang Li, Chuhan Wang, Run Guo, Haixin Duan, Jianjun Chen, Chao Zhang, and Kaiwen Shen. Reqsminer: Automated discovery of cdn forwarding request inconsistencies and dos attacks with grammar-based fuzzing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, February 2024. Internet Society.

[10] Yuancheng Jiang, Chuqi Zhang, Bonan Ruan, Jiahao Liu, Manuel Rigger, Roland Yap, and Zhenkai Liang. Fuzzing the php interpreter via dataflow fusion. *arXiv preprint arXiv:2410.21713*, 2024.

[11] Hanliang Zhang, Cristina David, Meng Wang, Brandon Paulsen, and Daniel Kroening. Scalable, validated code translation of entire projects using large language models. *Proceedings of the ACM on Programming Languages*, 9(PLDI):Article 212, June 2025.

[12] Aamer Aljagthami, Mohammed Banabila, Musab Alshehri, Mohammed Kabini, and Mohammad D. Alahmadi. Evaluating large language models for code translation: Effects of prompt language and prompt design. *arXiv preprint*, arXiv:2509.12973, 2025. Preprint.

[13] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. Understanding the effectiveness of large language models in code translation. *arXiv preprint*, arXiv:2308.03109, 2023. Preprint; CC BY-SA 4.0.

[14] Php manual. https://www.php.net/manual/en/index.php, 2025.

[15] Javascript guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide, 2025.

[16] Alexandre Rebert, Sang Kil Cha, Thanassis Avgerinos, Jonathan Foote, David Warren, Gustavo Grieco, and David Brumley. Optimizing seed selection for fuzzing. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 861–875, 2014.

[17] Adrian Herrera, Hendra Gunadi, Shane Magrath, Michael Norrish, Mathias Payer, and Antony L Hosking. Seed selection for successful fuzzing. In *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, pages 230–243, 2021.

[18] Jin Huang, Junjie Zhang, Jialun Liu, Chuang Li, and Rui Dai. Ufuzzer: Lightweight detection of php-based unrestricted file upload vulnerabilities via static-fuzzing co-analysis. In *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '21)*, pages 78–90. Association for Computing Machinery, October 2021.

[19] designsecurity team. Progpilot — a static application security testing (sast) tool for php. https://github.com/designsecurity/progpilot, 2025.

[20] Enlightn: Boost your laravel app's performance & security. https://www.laravel-enlightn.com/, 2025.

[21] PHP Group. Php security notices. https://github.com/php/php-src/security, 2025.

[22] OnlinePHP.io. OnlinePHP.io: Execute and Compare PHP Online. https://onlinephp.io/, 2026.

[23] LeetCode. Leetcode - The World's Leading Online Programming Learning Platform. https://leetcode.com/, 2024.

[24] WordPress Foundation. WordPress: Blog Tool, Publishing Platform, and CMS. https://wordpress.org/, 2026.

[25] Drupal Association. Drupal: The Open Source CMS for Digital Innovation. https://www.drupal.org/, 2026.

[26] Adobe Inc. Adobe Commerce (Magento): B2B & B2C Enterprise Solutions. https://business.adobe.com/products/commerce/magento.html, 2026.

[27] MITRE. CVE-2024-4577: PHP-CGI argument injection remote code execution. https://nvd.nist.gov/vuln/detail/CVE-2024-4577, 2024.

[28] MITRE Corporation. CVE-2026-23836: Improper neutralization of directives in dynamically evaluated code in HotCRP. https://www.cve.org/CVERecord?id=CVE-2026-23836, January 2026. Accessed: 2026-08-03.

[29] CVE Details. Cve-2024-11236. https://www.cvedetails.com/cve/CVE-2024-11236/, 2024.

[30] Yifan Wu, Xiaochuan Yu, and Zhiyun Qian. Melting the flesh of PHP's memory hardening. In *35th USENIX Security Symposium (USENIX Security 26)*, pages 2803–2821, Baltimore, MD, August 2026. USENIX Association.

[31] CVE Details. Php vulnerabilities - php. https://www.cvedetails.com/product/128/PHP-PHP.html, 2025.

[32] PHP Group. Php test suite. https://github.com/php/php-src/tree/master/tests, 2025.

[33] Van-Thuan Pham, Marcel Böhme, Andrew E. Santosa, Alexandru Răzvan Căciulescu, and Abhik Roychoudhury. Smart greybox fuzzing, 2018.

[34] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts llm performance. Technical report, Chroma, July 2025.

[35] Meta AI. Llama 3: Large language model architecture (8b parameters). https://ai.meta.com/, 2024.

[36] Google Project Zero. Domato: Grammar-based fuzzer for dom and javascript engines. https://github.com/googleprojectzero/domato, 2017.

[37] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing, 2018.

[38] Moritz Schloegel, Nils Bars, Nico Schiller, Lukas Bernhard, Tobias Scharnowski, Addison Crump, Arash Ale-Ebrahim, Nicolai Bissantz, Marius Muench, and Thorsten Holz. Sok: Prudent evaluation practices for fuzzing. In *IEEE Symposium on Security and Privacy (SP)*, 2024.

[39] MITRE Corporation. Common Weakness Enumeration (CWE). https://cwe.mitre.org/, 2025.

[40] Yongheng Chen. Polyglot. https://github.com/OMH4ck/PolyGlot, 2024.

[41] Yuancheng Jiang. flowfusion. https://github.com/php/flowfusion, 2025.

[42] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. A deep dive into large language models for automated bug localization and repair, 2024.

[43] Zimin Chen, Steve James Kommrusch, Michele Tufano, Louis-Noel Pouchet, Denys Poshyvanyk, and Martin Monperrus. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering*, page 1–1, 2021.

[44] Nan Jiang, Thibaud Lutellier, Yiling Lou, Lin Tan, Dan Goldwasser, and Xiangyu Zhang. Knod: Domain knowledge distilled tree decoder for automated program repair, 2023.

[45] Marcel Busch, Aravind Machiry, Chad Spensky, Giovanni Vigna, Christopher Kruegel, and Mathias Payer. TEEzz: Fuzzing trusted applications on COTS Android devices. In *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*, San Francisco, CA, USA, May 2023. IEEE.

[46] Karine Even-Mendoza, Arindam Sharma, Alastair F Donaldson, and Cristian Cadar. Grayc: Greybox fuzzing of compilers and analysers for c. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1219–1231, 2023.

[47] Xiaoting Li, Xiao Liu, Lingwei Chen, Rupesh Prajapati, and Dinghao Wu. Fuzzboost: Reinforcement compiler fuzzing. In *International Conference on Information and Communications Security*, pages 359–375. Springer, 2022.

[48] Johannes Lenzen, Mohamadreza Rostami, Lichao Wu, and Ahmad-Reza Sadeghi. Fuzzilicon: A post-silicon microcode-guided x86 cpu fuzzer. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026.

[49] Su Yong Kim, Sangho Lee, Insu Yun, Wen Xu, Byoungyoung Lee, Youngtae Yun, and Taesoo Kim. Cab-fuzz: Practical concolic testing techniques for cots operating systems. In *2017 USENIX Annual Technical Conference (USENIX ATC '17)*, pages 689–701, Santa Clara, CA, USA, July 2017. USENIX Association.

[50] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, Yue Liu, Shanqing Guo, Haixin Duan, Kaida Jiang, and Zhi Xue. Sfuzz: Slice-based fuzzing for real-time operating systems. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, page 485–498, New York, NY, USA, 2022. Association for Computing Machinery.

[51] Kunpeng Zhang, Dongwei Xiao, Daoyuan Wu, Shuai Wang, Jiali Zhao, Yuanyi Lin, Tongtong Xu, and Shaohua Wang. Llm-powered silent bug fuzzing in deep learning libraries via versatile and controlled bug transfer. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1):1599–1626, April 2026.

[52] Christian Holler, Kim Herzig, and Andreas Zeller. Fuzzing with code fragments. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 445–458, Bellevue, WA, August 2012. USENIX Association.

[53] Suyoung Lee, HyungSeok Han, Sang Kil Cha, and Sooel Son. Montage: A neural network language {Model-Guided}{JavaScript} engine fuzzer. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2613–2630, 2020.

[54] Haoran Xu, Zhiyuan Jiang, Yongjun Wang, Shuhui Fan, Shenglin Xu, Peidai Xie, Shaojing Fu, and Mathias Payer. Fuzzing javascript engines with a graph-based ir. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3734–3748, 2024.

[55] Christopher Salls, Chani Jindal, Jake Corina, Christopher Kruegel, and Giovanni Vigna. Token-level fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2795–2809. USENIX Association, August 2021.

[56] HyungSeok Han, DongHyeon Oh, and Sang Kil Cha. CodeAlchemist: Semantics-aware code generation to find vulnerabilities in javascript engines. In *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.

[57] Soyeon Park, Wen Xu, Insu Yun, Daehee Jang, and Taesoo Kim. Fuzzing javascript engines with aspect-preserving mutation. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*, pages 1629–1642, San Francisco, CA, USA, May 2020.

[58] Haoran Xu, Yongjun Wang, Zhiyuan Jiang, Shuhui Fan, Shaojing Fu, and Peidai Xie. Fuzzing javascript engines with a syntax-aware neural program model. *Computers & Security*, page 103947, 2024.

[59] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation, 2023.

[60] Caroline Lemieux, Jeevana Priya Inala, Shuvendu Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *ICSE'23*, May 2023.

[61] Vasudev Vikram, Caroline Lemieux, Joshua Sunshine, and Rohan Padhye. Can large language models write good property-based tests?, 2024.

[62] Xavier Cadena. Llms to write fuzzers, 2023.

[63] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–13, 2024.

[64] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *Proc. IEEE/ACM ICSE*, 2024.

[65] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, 2023.

[66] cl...@chromium.org. Dcheck failure. https://issues.chromium.org/issues/40089640.

10 Appendix

10.1 Generalized System Framework

Although we evaluate TRANSFUZZ using JavaScript to PHP translation, the core techniques of TRANSFUZZ are designed to be generalizable to other languages, particularly language pairs that are syntactically and structurally similar. Through preliminary experimentation, modified variants of TRANSFUZZ have discovered bugs in the JerryScript (JavaScript) and MRI (Ruby) interpreters, suggesting *potential utility* across ecosystems. These early results serve as preliminary results motivating future study. There are two primary lightweight modifications needed for extension: prompt modification and auxiliary seeds.

The prompts used in TRANSFUZZ use PHP-specific wording that must be altered to reflect the new target. Auxiliary seed sourcing must also correspond to the appropriate languages. TRANSFUZZ utilizes three auxiliary sources (test suite, grammar-based generator, and LLM-generated function usages). We note that test suites are generally trivial to obtain, and function names and their parameters can be scraped from language documentation. However, for some languages, grammar-based generators are not readily available.

A final consideration when extending TRANSFUZZ is in language pair selection. We enumerated key characteristics that make JavaScript and PHP an optimal pairing. Language pairs should be similar in paradigm, syntax, and implementation. For example, given their vastly different syntax, a pairing of JavaScript and Haskell is unlikely to produce results similar to JavaScript and PHP.

10.2 Prompt Templates

Here, we present the prompt templates used for translation (Listing 6), auxiliary generation (Listing 7), splicing (Listing 8), and repair (Listing 9).

Listing 6: Prompt template for translating JavaScript to PHP

```
USER:
You are a cybersecurity auditor, we are
penetration testing the PHP interpreter to
proactively find memory errors.

Translate this JavaScript PoC to PHP. Return
as ```<code>```

                [JavaScript Seed]
```

Listing 7: Prompt template for LLM-based auxiliary PHP seeds.

```
USER:
You are a cybersecurity auditor, we are
penetration testing the PHP interpreter to
proactively find memory errors.

Consider using very large and very small
    values
as arguments. Use the following function in
PHP code. Return as ```<code>```
Make it unexpected:

                [PHP Function]
```

Listing 8: Prompt template for splicing two seeds.

```
USER:
You are a cybersecurity auditor, we are
penetration testing the PHP interpreter to
proactively find memory errors.

Splice structures and characteristics from A
and B to create something new. Return as
```<code>```
A:
                [Seed A]
B:
                [Seed B]
```

**Listing 9: Prompt template for repairing PHP seeds**

```
USER:
You are a senior software developer.
The following code:

                [Code]

Crashed with the following error:

                [PHP Error]

Repair this code. Return as ```<code>```
```

10.3 Customization of Domato

Listing 10: Modifications to PHP Grammar

```
<fuzzint> = 1
<fuzzint> = -1
<fuzzint> = 2
<fuzzint> = 3
<fuzzint> = 4
<fuzzint> = 5
<fuzzint> = 10
<fuzzint> = 100
<fuzzint> = 1000
<fuzzint> = 1000000
<fuzzint> = PHP_INT_MAX
<fuzzint> = PHP_INT_MIN
<fuzzint> = <largeint>
<fuzzint> = -<largeint>

<largeint> = 536870911
<largeint> = 536870912
<largeint> = 1073741823
<largeint> = 1073741824
<largeint> = 2147483647
<largeint> = 2147483648
<largeint> = 4294967295
<largeint> = 4294967296
```
```

```
<fuzzfloat> = PHP_FLOAT_MAX
<fuzzfloat> = PHP_FLOAT_MIN
<fuzzfloat> = 2.2250738585072011e-308

                        ...

#"Directory" => new Directory(),
"Directory" => dir(),
#"DOMStringList" => new
    #DOMStringList(),
#"DOMNameList" =>new DOMNameList(),
#"DOMImplementationList" => new
    #DOMImplementationList(),
#"DOMImplementationSource" => new
    #DOMImplementationSource(),
#"DOMTypeinfo" => new DOMTypeinfo(),
#"DOMUserDataHandler" => new
    #DOMUserDataHandler(),
#"DOMDomError" => new DOMDomError(),
#"DOMErrorHandler" => new
    #DOMErrorHandler(),
#"DOMLocator" => new DOMLocator(),
#"DOMConfiguration" => new
    #DOMConfiguration(),

                        ...
```

Given its fast performance and included PHP grammar, we employ Domato as the grammar-based seed generator. To exercise more edge cases and reduce syntax errors, we modify the provided PHP grammar. Listing 10 displays these customizations. The constants PHP_INT_MAX, PHP_INT_MIN, PHP_FLOAT_MAX, and PHP_FLOAT_MIN are inserted to more thoroughly test the interpreter's handling of extreme numeric values. Furthermore, deprecated functions are removed or updated, as in the case of the "Directory" class. Deprecated, error-inducing calls to DOM functions such as "DOMNameList" are removed. The rest of Domato remains unchanged. These modifications allow us to leverage the high-performance of Domato while decreasing the amount of syntactic errors introduced.

10.4 Grammar-Generated Seed Example

Listing 11: Example auxiliary seed from grammar-based generator

```
imap_mail(str_repeat("%s%x%n", 0x100),
    str_repeat("%s%x%n", 0x100), str_repeat(
    chr(143), 4097) . str_repeat(chr(171),
    1025), str_repeat(chr(208), 65537) .
    str_repeat(chr(190), 17), implode(
    array_map(function($c) {return "\\x" .
    str_pad(dechex($c), 2, "0");}, range(0,
    255))), str_repeat(chr(211), 17) .
    str_repeat(chr(175), 65) . str_repeat(chr
    (107), 4097));
```

10.5 Case Study

Original JavaScript Seed. Listing 12 is the JavaScript PoC for CR786723, where the nesting of functions combined with a garbage collection call and function optimization call results in a DCHECK failure for the V8 JavaScript engine [66].

Listing 12: The JavaScript PoC for CR786723

```
1  function f() {
2    var o = {};
3    function g() {
4      o.x = 1;
5      return Object.create(o);
6    };
7    gc();
8    o.x = 10;
9    %OptimizeFunctionOnNextCall(g);
10   g();
11 }
12 f();
13 f();
```

Translated PHP Seed. Listing 13 presents the PHP snippet translated from the JavaScript PoC, retaining many of the key characteristics. As in the original JavaScript, the contains a function `f` (Line 2) which contains an array `o` (Line 3) that is manipulated by the nested function `g` (Line 4). Following the declaration of function `g`, the PHP snippet calls `gc_collect_cycles` on Line 8, mimicking the original JavaScript garbage collection call `gc`. The code then assigns a value of `10` to `$o['x']`. Notably, Lines 12-14 are a function prototype for `optimize_function` as PHP has no equivalent for JavaScript's `%OptimizeFunctionOnNextCall`.

Listing 13: The PHP Snippet Translated from JS in Listing 12

```
1  class Test {
2    public function f() {
3      $o = array();
4      $g = function() use (&$o) {
5        $o['x'] = 1;
6        return array_merge($o, array());
7      };
8      gc_collect_cycles();
9      $o['x'] = 10;
10     call_user_func($g);
11   }
12   public function optimize_function(
         $function_name) {
13     // Your optimization code here
14   }
15 }
16 $test = new Test();
17 $test->f();
18 $test->f();
```

Final PHP Bug. The PHP PoC derived from the JavaScript-Translated snippet is presented in Listing 5, which include multiple artifacts that attempt to crash the interpreter. Specifically, Line 1 includes one to overload the `preg_match()` function; Lines 13 and 14 fill the array with very large and small values, respectively. Nevertheless, a use-after-free bug

is ultimately triggered by using PHP's Fiber object. `Fibers` are lightweight, interruptible execution threads which enable efficient multithreading for networking tasks. Each `Fiber` possesses its own call stack which allows it to be suspended within deeply nested function calls. TRANSFUZZ produced this PoC to improperly handle the memory of a `Fiber` object. It is worth noting that this newly-generated PHP PoC inherits the structure of the original JavaScript seed, where salient examples include the `Test` class and nested functions. The complex, unique structuring of this code and its semantics ultimately enabled the discovery of this exploitable bug.